

Experiment -9

AIM- To aggregate data from multiple on-board sensors and format it as structured telemetry frame.

APPARATUS- STM32 Discovery Kit, STM32 CUBE IDE, STM32 CubeMX, Realterm.

PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L475VGT6.
3. **Clock Selection:** Go to RCC and configure clock as “By pass Clock Source” as shown below:

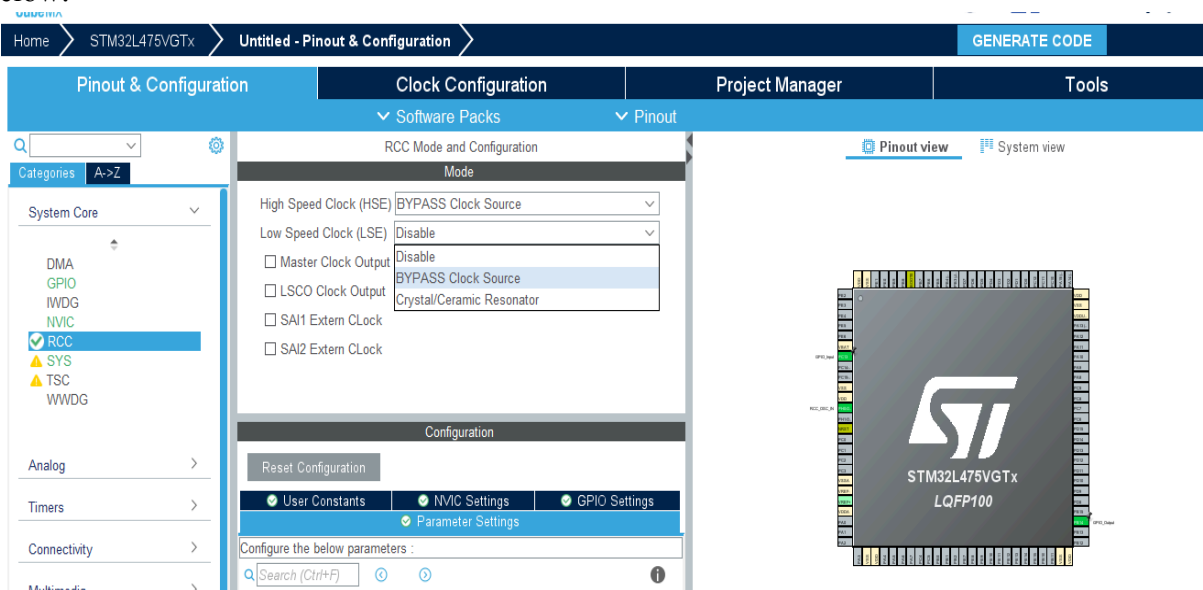


Fig-9.1 – Clock Selection

4. As per user manual of board ST LINK is internally connected to PB6(Tx) and PB7(Rx) of USART1.

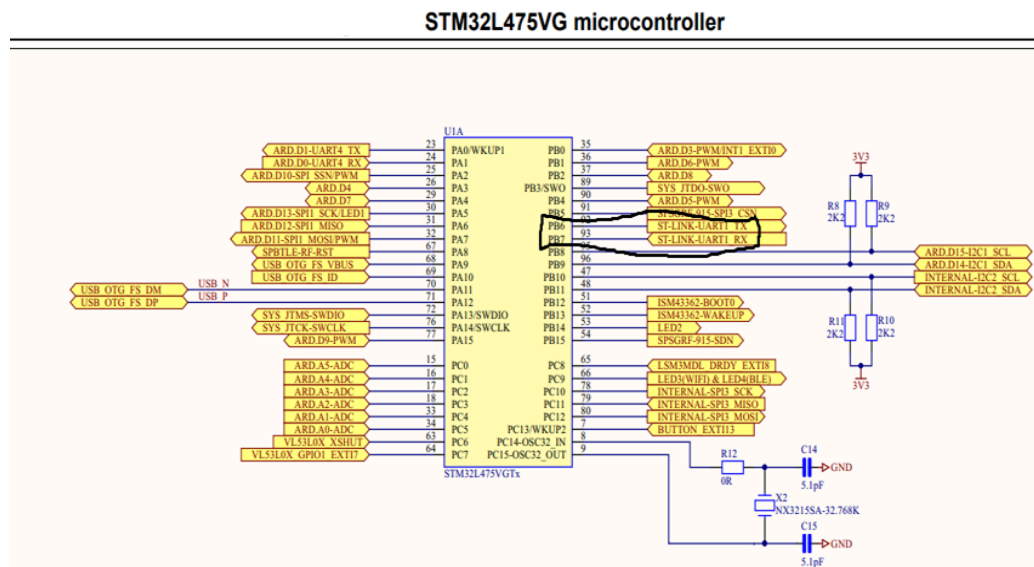


Fig-9.2 – STM32 Board Pin Configuration

- Under “connectivity” click on “USART1” and make PB6 as USART1_Tx and PB7 as USART2_Rx as shown below:

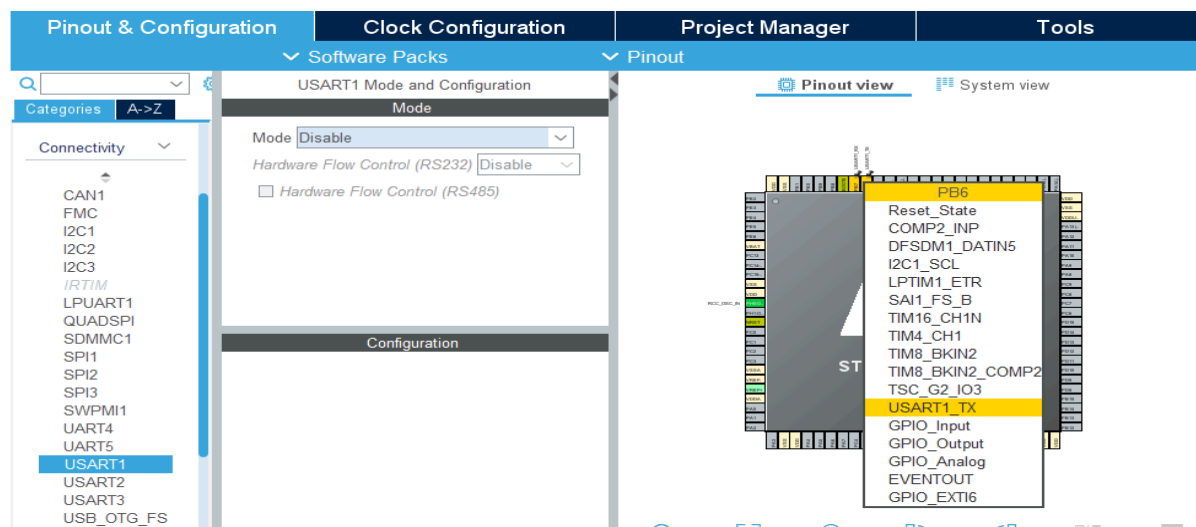


Fig-9.3 – STM32 Board Pinout and Configuration

- Select USART1 and click on “Asynchronous” in “Mode” as shown below :

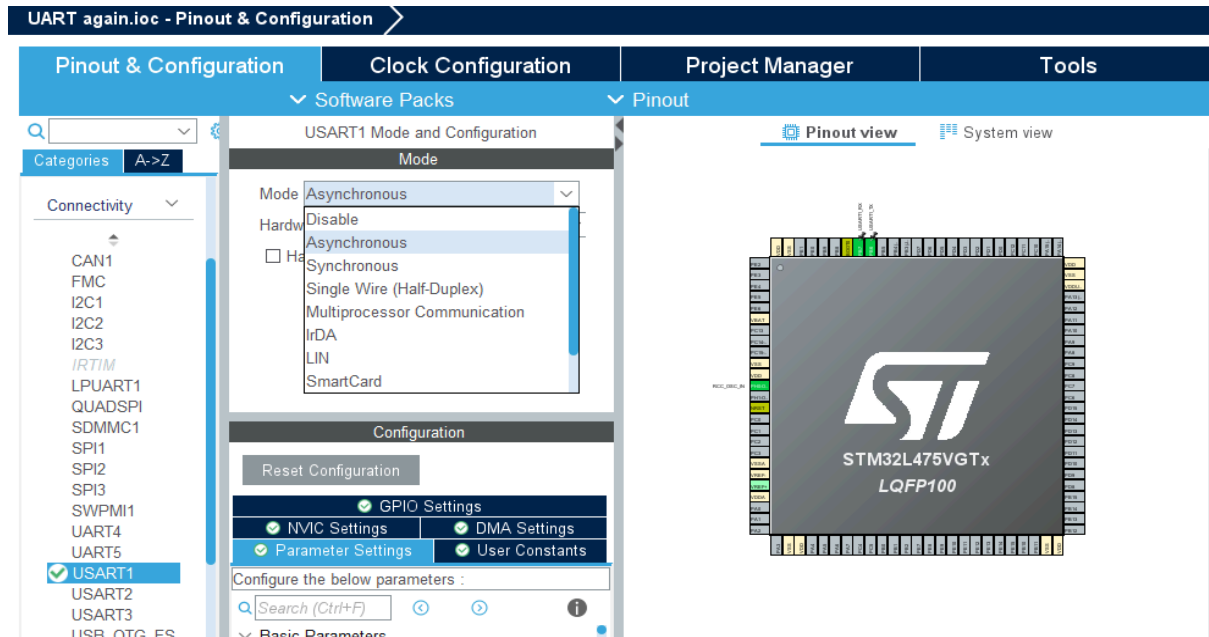


Fig-9.4 – Selection of Asynchronous mode

7. Go to “connectivity” and also select I2C1 in mode.
8. Click on PB8 and select I2C1_SCL to make PB8 pin as clock source pin as shown below:

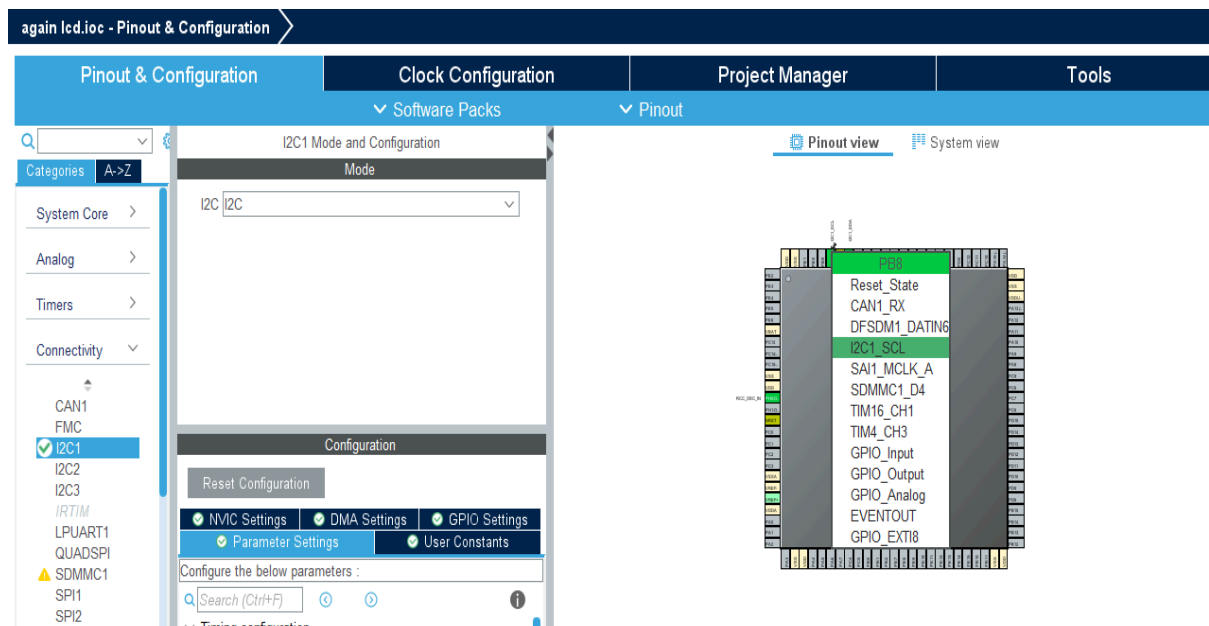


Fig-9.5 – Settings of Basic Parameters

9. **Clock Configuration:** Configure the clock as shown below to get a maximum frequency of 80MHz.

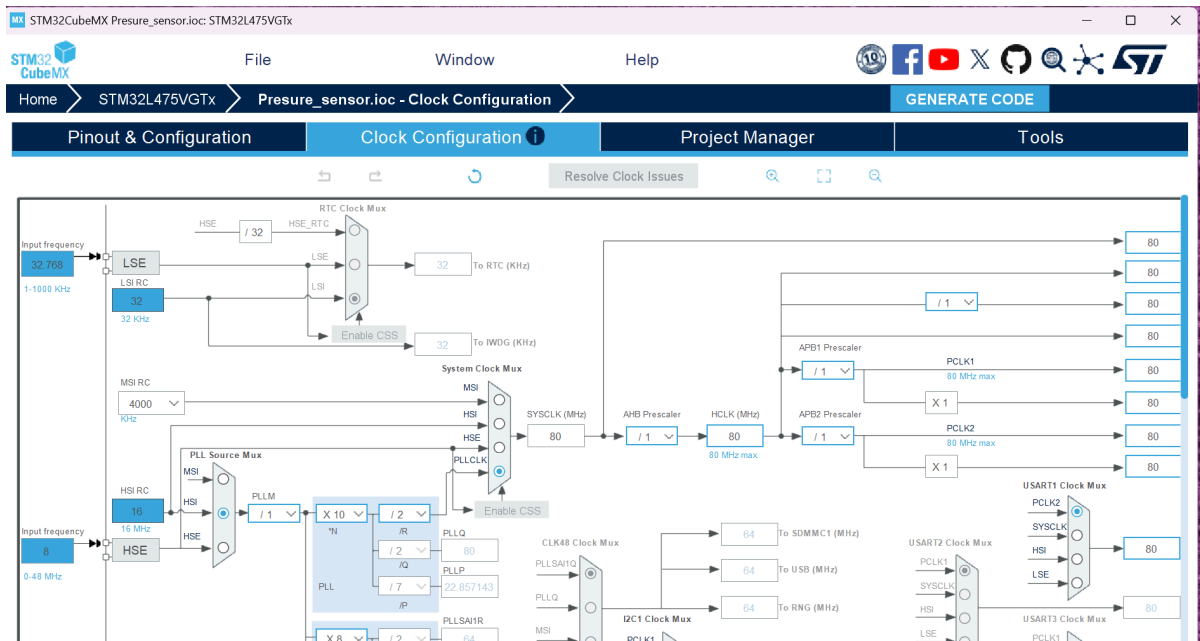


Fig-9.6 – Clock Configuration

10. Write Project name and select CubeIDE.
11. Click on generate code.
12. Click on open project.
13. Click 'OK' on the generated popup showing “successfully imported the project on the work space” and the project will be successfully added to the cube IDE.
14. Copy BSP drivers to your project.
15. The last version of the STM32CubeL4 package is downloaded by default in the STM32CubeMX repository
(C:\Users\user_name\STM32Cube\Repository\STM32Cube_FW_L4_Vx.xx.x\Drivers).

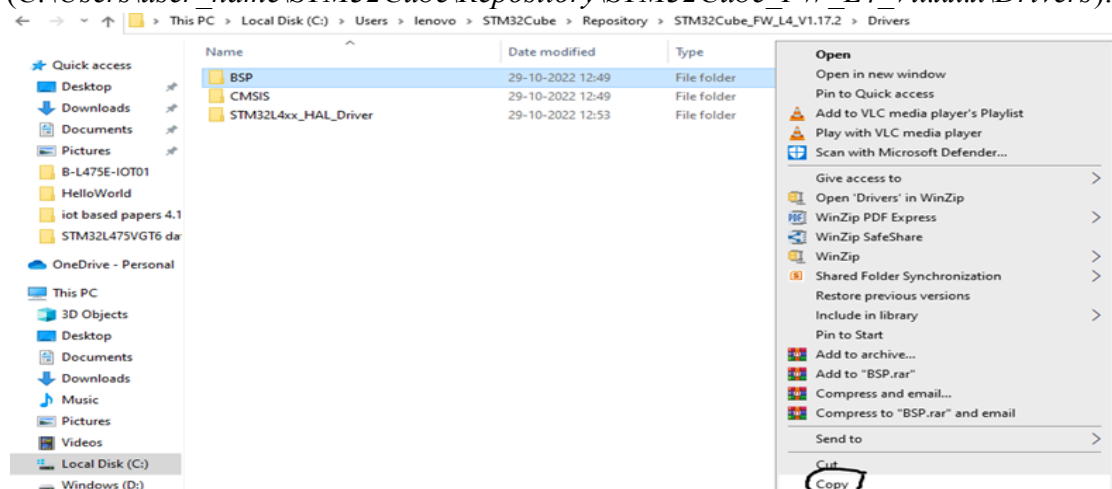


Fig-9.7 – BSP drivers location

16. Paste the copied BSP drivers in the drive folder as shown below:

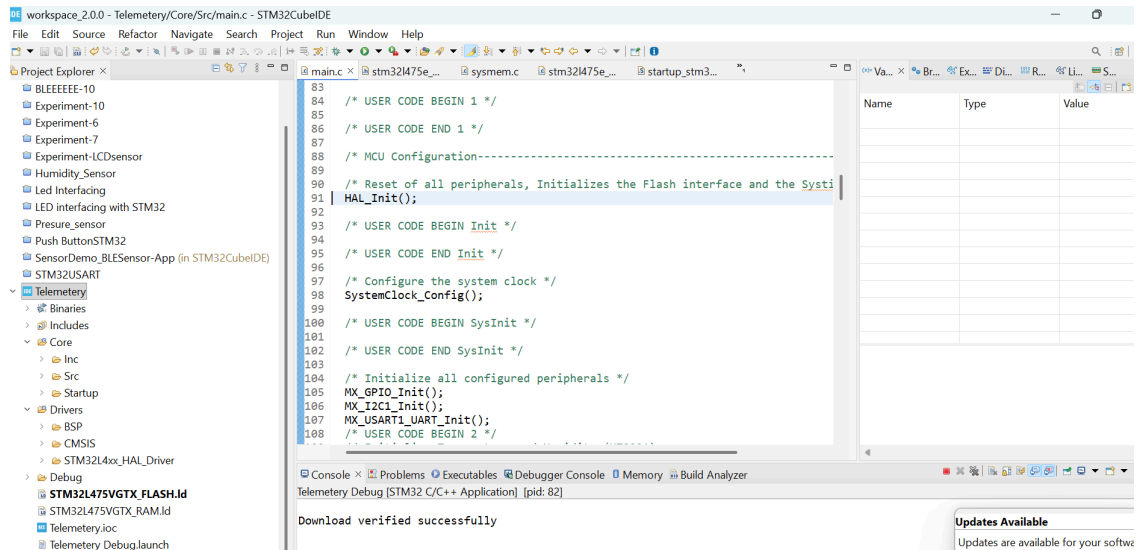


Fig-9.8 – Paste BSP drivers in project folder

17. Delete all folders in BSP, except B-L475E-IOT01 and the component as shown:

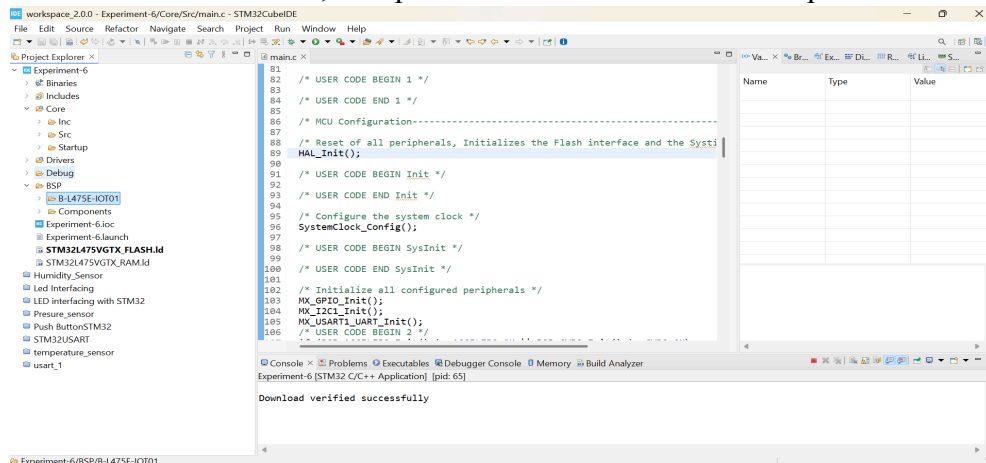


Fig-9.9 – B-L475E-IOT01 and component folder in BSP drivers

18. Keep only these folders in B-L475E-IOT01 and the component as shown in figure:

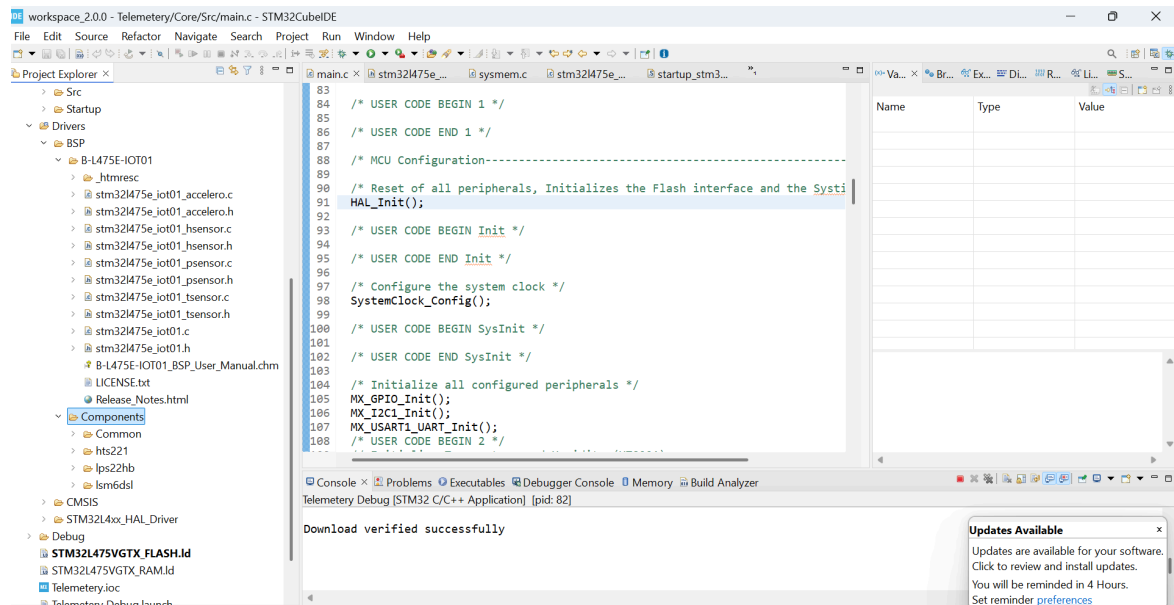


Fig-9.10 – Folders in B-L475E-IOT01 and Component

19. Update include paths: From Project menu or File menu, go to Properties > C/C++ Build > Settings > Tool Settings > MCU GCC Compiler > Include paths and click “add” icon.png to include the new paths as shown below:

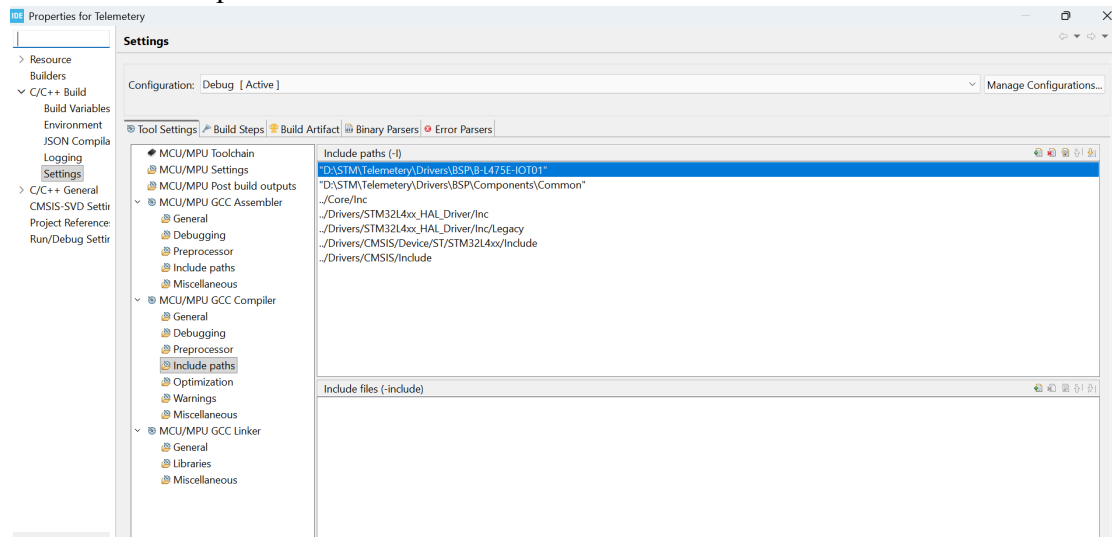


Fig-9.11 – Path settings

20. Add ../Drivers/BSP/B-L475E-IOT01 and ../Drivers/BSP/Components/lsm6dsl paths as addresses given below:
C:\Users\lenovo\STM32Cube\Repository\STM32Cube_FW_L4_V1.17.2\Drivers\BSP\B-L475E-IOT01“ and
C:\Users\lenovo\STM32Cube\Repository\STM32Cube_FW_L4_V1.17.2\Drivers\BSP\Components\lsm6dsl

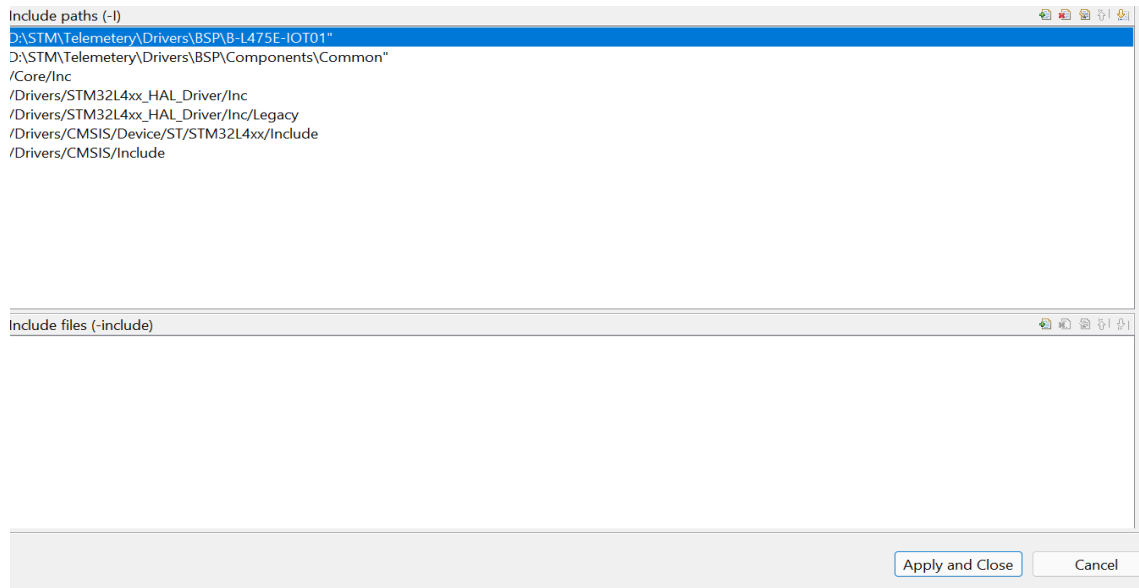


Fig-9.12 – Folders Path settings

21. After adding both path click “ apply and close”.
22. Go to properties of project>go to settings>go to MCU/MPU settings>Click to float as >apply and close as shown in figure:

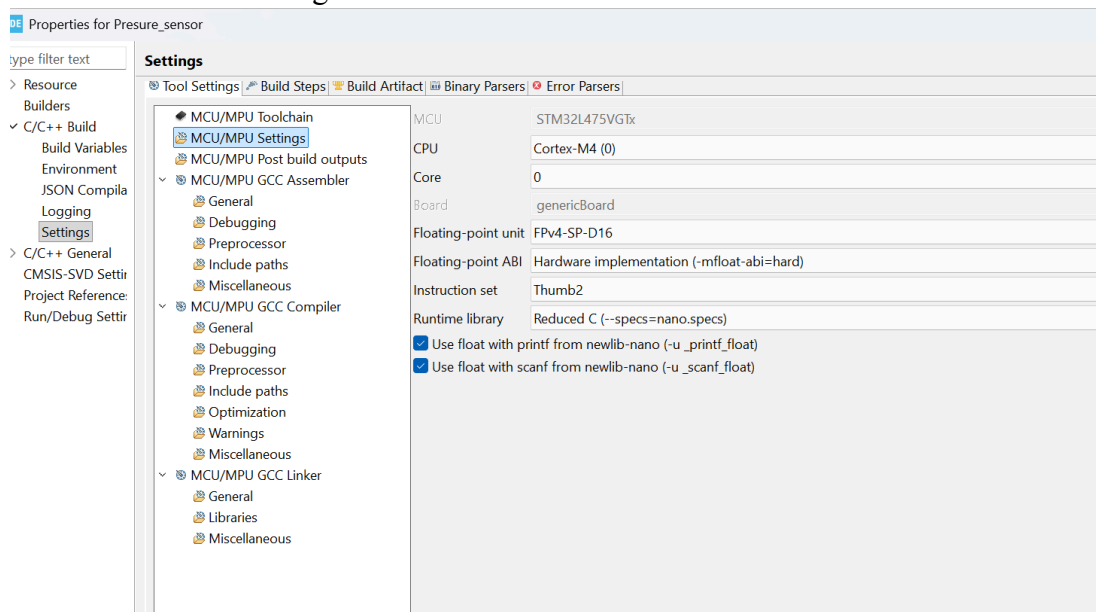


Fig-9.13 – MCU/MPU setting

23. Then “Build” and go to main.c file.
24. Include the header files in main.c files.

```
/* Includes */  
#include "stm32l475e_iot01.h"  
#include "stm32l475e_iot01_hsensor.h"
```

```
#include "stm32l475e_iot01_tsensor.h"
#include "stm32l475e_iot01_psensor.h"
#include "stm32l475e_iot01_accelero.h"
#include "stdlib.h"
#include <math.h>
#include <string.h>
#include <stdio.h>
```

25. Write the following instructions:

```
/* USER CODE BEGIN PV */
char str[150]; // Increased size to accommodate more data
float temp_value = 0;
float hum_value = 0;
float press_value = 0; // Added for Pressure
int16_t xyz_acc[3] = {0}; // Added for Accelerometer (X, Y, Z)
uint8_t msg_init[] = "=====> All Sensors Initialized \r\n";
/* USER CODE END PV */
```

26. Write the following instructions as given:

```
/* USER CODE BEGIN 2 */
// Initialize Temperature and Humidity (HTS221)
if (BSP_HSENSOR_Init() != HSENSOR_OK) {
    HAL_UART_Transmit(&huart1, (uint8_t*)"HSENSOR Error\r\n", 15, 1000);
}
BSP_TSENSOR_Init();

// Initialize Pressure Sensor (LPS22HB)
if (BSP_PSENSOR_Init() != PSENSOR_OK) {
    HAL_UART_Transmit(&huart1, (uint8_t*)"PSENSOR Error\r\n", 15, 1000);
}

// Initialize Accelerometer (LSM6DSL)
if (BSP_ACCELERO_Init() != ACCELERO_OK) {
    HAL_UART_Transmit(&huart1, (uint8_t*)"ACCELERO Error\r\n", 16, 1000);
}

HAL_UART_Transmit(&huart1, msg_init, sizeof(msg_init), 1000);

/* USER CODE END 2 */
```

27. Write the following instructions in **while (1) loop** under user code begin 3.

```
// 1. Read Humidity and Temperature
hum_value = BSP_HSENSOR_ReadHumidity();
temp_value = BSP_TSENSOR_ReadTemp();
// 2. Read Pressure (returns value in hPa/mbar)
press_value = BSP_PSENSOR_ReadPressure();
// 3. Read Accelerometer (returns values in mg - milli-g)
BSP_ACCELERO_AccGetXYZ(xyz_acc);
```



```
// Format the output string
// T: Temp, H: Humidity, P: Pressure, Acc: X, Y, Z
snprintf(str, sizeof(str), "T:%.1fC | H:%.1f%% | P:%.1f hPa | Acc: X:%d Y:%d Z:%d\r\n",
temp_value, hum_value, press_value, xyz_acc[0], xyz_acc[1], xyz_acc[2]);
HAL_UART_Transmit(&huart1, (uint8_t*)str, strlen(str), 1000);
HAL_Delay(1000);
```

28. Now connect the board and click on “Build”, “debug”.
29. Click on “Switch” on popup and click on “Resume”, the program will start execute.
30. Now, open the console “Real term” and set the baud rate as used in program (in this program it is 115200), select port (as per use) and click on “change” on the real term console.

OBSERVATION: The output values of different sensors will display on real term as shown below:

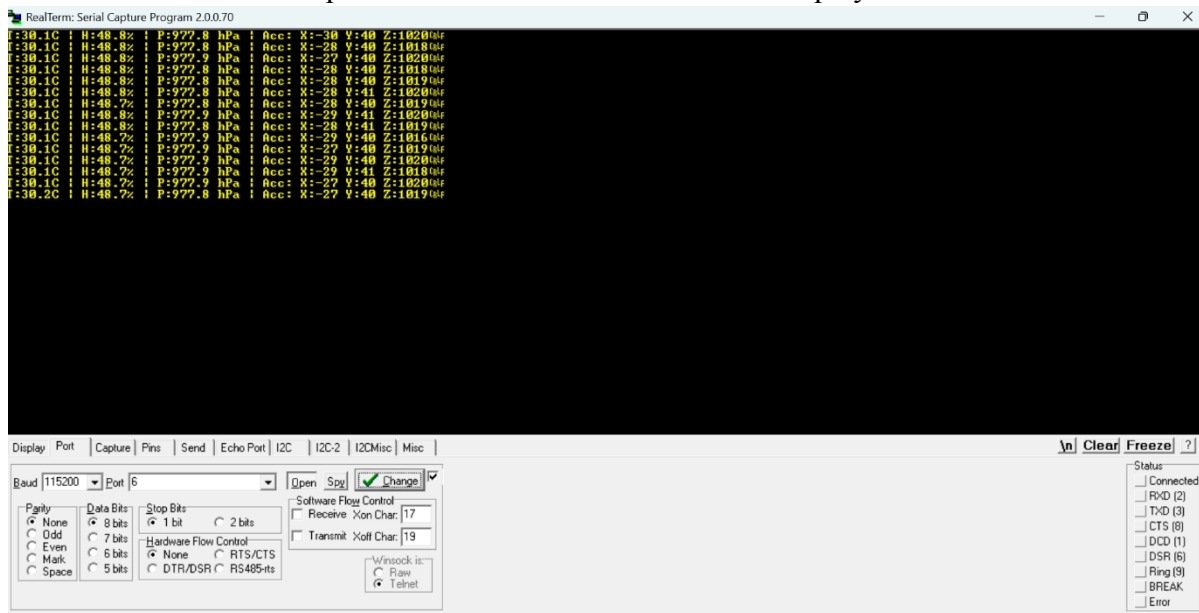


Fig: 9.14– Output from experiment

Reflection Summary

1. How did configuring USART1, I2C1, and clock settings in STM32CubeMX help you understand the complete workflow of setting up a multi-sensor telemetry system on the STM32L475 IoT node?

Configuring USART1, I2C1, and clock settings in STM32CubeMX helped me understand the complete workflow of building a multi-sensor telemetry system on the STM32L475 IoT node. USART1 enabled serial communication between the STM32 board and RealTerm for displaying sensor data, while I2C1 allowed communication with onboard sensors such as humidity, temperature, pressure, and accelerometer modules. Configuring the clock to 80 MHz ensured proper timing and stable peripheral operation. Through this process, I learned how hardware peripherals, communication protocols, and software configuration work together in an embedded

IoT system.

2. What challenges did you encounter while integrating the BSP sensor drivers and updating the include paths for humidity, temperature, pressure, and accelerometer sensors in STM32CubeIDE, and how did you overcome them?

While integrating the BSP sensor drivers and updating the include paths in STM32CubeIDE, I faced issues such as missing header files, compilation errors, and incorrect driver paths. Some drivers were not recognized because the required BSP folders and component files were not properly added to the project settings. I solved these issues by copying the correct BSP drivers, keeping only the necessary folders, and updating the include paths under the MCU GCC Compiler settings. Rebuilding the project after correcting the paths resolved the errors and allowed successful sensor initialization and data acquisition.

3. In what ways does combining all sensor readings into a single structured UART telemetry frame and monitoring it on RealTerm enhance your understanding of real-time data acquisition in IoT applications?

Combining all sensor readings into a single structured UART telemetry frame improved my understanding of real-time data acquisition in IoT applications. By formatting temperature, humidity, pressure, and accelerometer values into one readable string and transmitting it continuously through UART, I learned how multiple sensor outputs can be synchronized and monitored efficiently. Observing the telemetry data on RealTerm demonstrated how embedded systems collect, process, and transmit real-time environmental data for IoT monitoring applications. It also showed the importance of structured data formatting for debugging, analysis, and communication between devices.